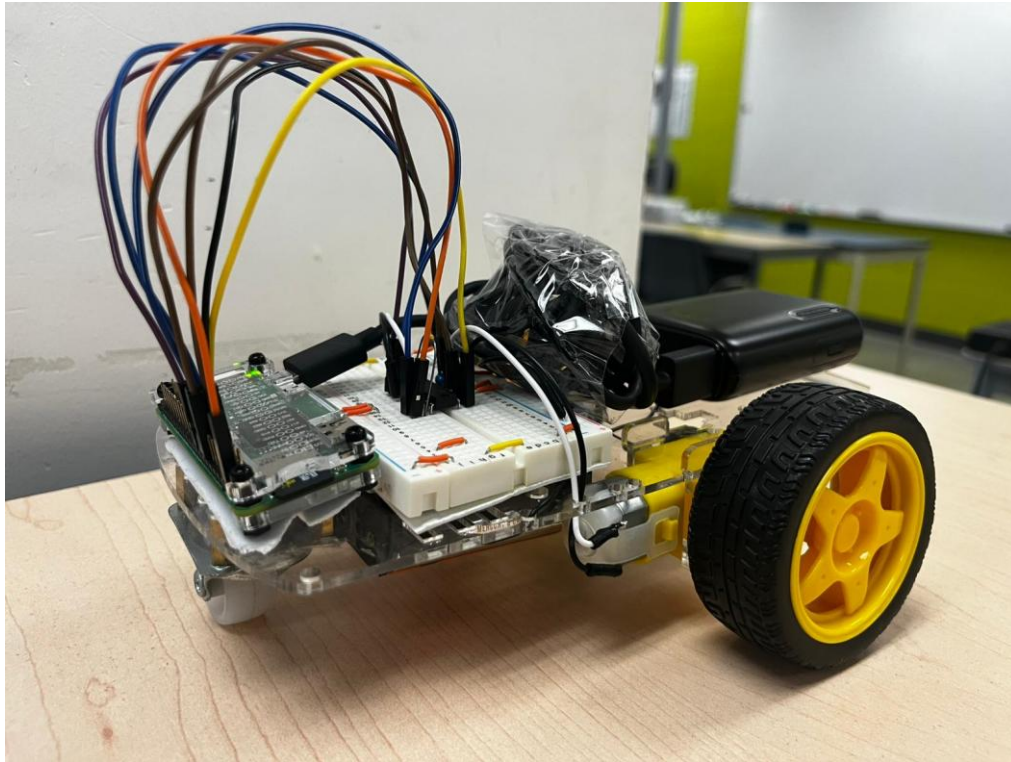


RoboRide Project Report



Group 2

Bilal Hussain - #9042866

Nikunj Patel - #9047741

Engineering Project – 3

Safat Khan

Reda Fayek

Submission date: - 10/08/2025



1. Abstract

The *RoboRide* project focuses on the development of a Raspberry Pi–controlled robotic car capable of forward, backward, and rotational movements with variable speed control. The objective was to design a low-cost, reliable, and educational robotics platform that connects theoretical learning with practical embedded system applications. The system integrates a Raspberry Pi, L293D motor driver IC, and dual DC motors mounted on a two-wheel chassis, powered by a 5000 mAh power bank and a 6 V AA battery pack.

The control program, written in C using the WiringPi and SoftPWM libraries, generates precise PWM signals to regulate direction and speed. Through systematic debugging and iterative testing, challenges such as wiring congestion, polarity mismatches, and motor startup hesitation were successfully resolved. The robot achieved smooth, stable motion across all directions, confirming the effectiveness of its design.

Overall, *RoboRide* demonstrates a successful application of embedded systems and motor control principles. It also highlights strong potential for use in STEM education, autonomous mobility, and small-scale delivery systems, forming a foundation for future innovation in intelligent robotic platforms.

Table of Contents

1. Abstract.....	2
3. Introduction	5
3.1 Project Background	5
3.2 Project Objectives:	5
3.3 Scope:.....	5
3.4 Overview	6
4. Project Requirements	7
4.1 Functional Requirements	7
4.2 Non-Functional Requirements	8
4.3 Technical Requirements	9
5. Materials and Components	10
5.1 Hardware Components	10
5.2 Cost Analysis	11
5.3 Software Tools	12
5.4 Software Environment Setup.....	13
5.5 Setup Procedure	13
6. Software Requirements Specification (SRS)	14
7. Software Design Description (SDD).....	17
8. Circuit Design	24
8.1 Circuit Diagram	24
8.2 Schematic Diagram	25
8.3 Imagery	26
8.4 Wiring Description.....	27
9. Implementation.....	28
9.1 Code Structure	28
9.2 Explanation of code for better understanding	29
9.3 Key Code Snippets	30
9.4 Challenges and Solutions	32
10. Testing and Results	33
10.1 Testing Plan.....	33
10.2 Test Results	37
10.3 Observations	38

RoboRide Report – Bilal Hussain – Nikunj Patel

11. Discussion	39
11.1 Analysis of Results	39
11.2 Limitations.....	39
11.3 Potential Improvements.....	39
12. Conclusion	40
13. References.....	41
14. Appendices.....	42
14.1 Full Code Listing.....	42
14.2 Additional Diagrams or Tables	46

3. Introduction

3.1 Project Background

- Automation and robotics are rapidly transforming how industries and institutions approach short-distance transportation. The *RoboRide* project was conceptualized to design a simple yet effective prototype that demonstrates mobility control through embedded systems and motor driver integration. Inspired by industrial delivery robots and educational robotics kits, this project bridges theoretical knowledge with hands-on engineering application. Using a Raspberry Pi as the control unit, the system showcases real-world motor actuation and power management through pulse-width modulation (PWM). The motivation behind this project was to gain a deeper understanding of electronic interfacing, motion control, and hardware-software synchronization—skills that are crucial in modern electronic systems engineering.

3.2 Project Objectives:

- Develop a functional Raspberry Pi-based robotic car capable of controlled movement.
- Implement direction and speed control using an L293D motor driver and PWM signals.
- Integrate hardware components including motors, batteries, and wiring on a chassis platform.
- Test and validate performance through motion experiments and oscilloscope measurements.
- Analyze challenges related to wiring congestion, motor polarity, and power supply limitations.

3.3 Scope:

- The scope of this project includes the design, wiring, and software implementation of a robotic vehicle that can move forward, backward, left, and right under manual command. The system utilizes a Raspberry Pi for control logic and an L293D driver for motor interface. The project focuses on establishing reliable wired control, power management, and PWM testing.
- However, it does not cover autonomous navigation, wireless communication, obstacle detection, or sensor integration. These features are identified as possible future expansions that would increase the robot's intelligence and adaptability.

3.4 Overview

This report is structured to cover all aspects of the *RoboRide* project in a clear and logical sequence:

- **Introduction**
Provides the project background, objectives, and scope of the *RoboRide* system, highlighting its relevance to industrial transport and STEM education.
- **Project Requirements**
Outlines the functional, non-functional, and technical requirements that guided the development of the robotic vehicle.
- **Materials and Components**
Details all hardware components such as the Raspberry Pi, L293D motor driver, power supplies, and DC motors, along with cost analysis and software tools used.
- **Software Requirements Specification (SRS)**
Describes the system architecture, software functionalities, user interaction through terminal commands, and project constraints.
- **Software Design Description (SDD)**
Explains the code architecture, data handling, and modular design implemented for motion control, PWM speed regulation, and system safety.
- **Circuit Design**
Presents detailed wiring diagrams, pin configurations, and descriptions of the hardware setup integrating motors, driver IC, and power sources.
- **Implementation**
Covers the code structure, core functionalities, key command logic, and problem-solving strategies adopted during programming and debugging.
- **Testing and Results**
Documents the experimental validation of PWM signals, motor performance, and overall system functionality with expected versus actual outcomes.
- **Discussion**
Analyzes testing results, evaluates system efficiency, identifies limitations such as wiring congestion and torque loss, and reflects on performance during the demonstration.
- **Conclusion & Future Work**
Summarizes the project achievements and outlines future enhancements such as wireless control, sensor integration, and AI-based obstacle detection.
- **References & Appendices**
Lists all cited materials, code listings, wiring diagrams, oscilloscope readings, and supplementary documentation supporting the project.

4. Project Requirements

4.1 Functional Requirements

The *RoboRide* system is designed to perform essential motion and control functions through the integration of hardware and software components. The primary functional requirements are as follows:

- **Motor Control and Movement**
The system must control two DC motors using the L293D motor driver to achieve movement in all directions — forward, backward, left, and right — based on user commands entered via the Raspberry Pi terminal.
- **Speed Control through PWM**
The Raspberry Pi must generate pulse-width modulation (PWM) signals to vary the motor speeds between slow and fast modes, ensuring smooth acceleration and efficient power usage.
- **User Command Input and Processing**
The system must accept user inputs through a terminal interface and execute corresponding movement commands in real time. Commands include FWD, BCK, LFT, RGT, STOP, and speed control (1, 2).
- **System Safety and Motor Protection**
Before executing a new command, the program must stop both motors to avoid sudden directional changes or electrical stress on the motor driver circuit.
- **Power Management**
The design must support dual power inputs — a 5000 mAh power bank for the Raspberry Pi and 4 AA batteries for the motors — to ensure stable operation without voltage drops.
- **Expandability for Future Functions**
The system should be designed with flexibility to support additional features such as sensor integration, Bluetooth communication, or autonomous navigation in future iterations.

4.2 Non-Functional Requirements

The *RoboRide* system must not only perform its intended functions but also meet key standards of performance, reliability, and usability. The following non-functional requirements ensure that the robotic vehicle operates efficiently, safely, and consistently under typical use conditions:

- **Performance Requirements**

- The robot must respond to user commands within one second of input through the terminal interface.
- Motor speed transitions between slow and fast modes should occur smoothly without sudden jerks.
- PWM duty cycles must remain stable to maintain consistent motor speed during extended operation.

- **Reliability Requirements**

- The system must operate continuously for at least 30 minutes on a full charge without overheating or voltage drops.
- All motor control functions must remain operational even after repeated start-stop cycles.
- Dual power sources must provide stable voltage levels, ensuring the Raspberry Pi and motors do not interfere with each other.

- **Usability Requirements**

- The command interface should be simple and intuitive, allowing users to operate the robot without prior programming experience.
- Command responses and movement feedback should be easily observable, providing users with clear indication of successful execution.
- Wiring and component layout should allow easy maintenance, replacement, or system upgrades.

- **Efficiency Requirements**

- The design must optimize power consumption by using PWM to reduce motor load and battery drain.
- The Raspberry Pi should handle processing efficiently without overutilizing CPU resources or causing thermal issues.

4.3 Technical Requirements

Category	Requirement Description
Platform Requirements	Must operate on a Raspberry Pi running Raspberry Pi OS (Linux), powered via a 5V power bank or USB source.
Interface Requirements	Must interface with the L293D motor driver to control two DC motors for movement and rotation.
Power System Requirements	The system must use a dual power setup, with the Raspberry Pi powered separately from the motors to prevent voltage fluctuation.
Control Input Requirements	Motion commands (FWD,BCK,LFT,RGT & STOP) must be entered through a terminal-based command line interface (CLI).
Output Requirements	The robot should respond to user commands by performing corresponding movements with visible motor actions.
Speed Control Requirements	The system must implement PWM-based speed control with at least two preset levels (slow and fast).
Wiring and Circuit Requirements	Proper GPIO pin mapping and common ground must be maintained for stable circuit operation.
Safety Requirements	A 100 ms delay must be included between direction changes to protect the L293D IC and prevent back current.
Documentation Requirements	Include detailed setup instructions, wiring schematic, code listing, and component list for replication.
Future Expansion	System should support integration of additional modules such as ultrasonic sensors, Bluetooth, or line-following capabilities.

5. Materials and Components

5.1 Hardware Components

The *RoboRide* prototype integrates several electronic and mechanical components to achieve motor control, power management, and system stability. The table below lists all hardware components used in the project.

No.	Components	Pictures	Qty	Description
1	Raspberry pi 0 W		1	Acts as the brain of the system, processing input commands and generating PWM signals to control motor speed and direction.
2	L293D Motor Driver IC		1	Serves as the interface between the Raspberry Pi and the DC motors, providing the necessary current and voltage to drive the motors.
3	DC Motors		2	Provide motion to the robotic vehicle; connected to the driver for bidirectional control.
4	Car Chassis		1	Mechanical base that holds the motors, driver, and Raspberry Pi; enables movement on flat surfaces.
5	Breadboard		1	Used for prototyping and connecting electronic components without soldering.
6	Jumper Wires		~20	Facilitate electrical connections between the Raspberry Pi, motor driver, and other components.
7	Power Bank (5000mah)		1	Provides a stable 5v power supply to the Raspberry Pi.
8	4 * AA Battery pack		1	Powers the DC motors through the motor driver IC, ensuring isolated motor operation.
9	Wheels/Tyres		2	Convert motor rotation into linear motion for vehicle movement.
10	USB – Micro USB cable		1	Used to connect and power the Raspberry Pi from the power bank.

5.2 Cost Analysis

No.	Component	Estimated Cost (CAD)
1	Raspberry Pi Zero W	60.00
2	L293D Motor Driver IC	2.00
3	DC Motors (x2)	10.00
4	2WD Car Chassis	12.00
5	Breadboard	6.00
6	Jumper Wires Set	5.00
7	Power Bank (5000 mAh)	20.00
8	AA Battery Holder	3.00
9	AA Batteries (x4)	4.00
10	USB Cable (Micro-USB)	4.00
11	Miscellaneous Hardware (nuts, bolts, spacers)	5.00
12	Laptop (for programming)	800.00

Note:

- The laptop cost applies only to initial development and is not a recurring material expense.
- The total hardware cost (excluding laptop) for the prototype is approximately 131 CAD.

Total Cost: \$131/- only

5.3 Software Tools

The *RoboRide* project made use of several software tools and libraries for programming, simulation, circuit design, and testing. These tools were selected for their accessibility, reliability, and compatibility with the Raspberry Pi environment. The table below summarizes each tool, its purpose, license type, and cost.

No.	Software Tool	Description	License Type	Cost	Icon
1	Terminal Interface	Used to write, compile, and execute C programs directly on the Raspberry Pi. Also provides debugging features for testing motor commands and PWM control.	Open Source (MIT License)	Free	
2	WiringPi Library	A GPIO control library that enables communication between the Raspberry Pi and external components such as the L293D motor driver.	Open Source (GNU GPLv3)	Free	WiringPi
3	SoftPwm Library	Provides software-based Pulse Width Modulation (PWM) functionality for the Raspberry Pi, used for controlling motor speed.	Open Source (GNU GPLv3)	Free	SoftPwm
4	Raspberry pi OS	The official operating system for Raspberry Pi, used to execute code, manage GPIO, and handle power and process control.	Open Source (GNU GPLv2)	Free	 Raspberry Pi
5	Fritzing	Used to design and simulate the circuit diagram before physical implementation, ensuring accurate wiring of the L293D motor driver and Raspberry Pi pins.	Open Source (GPL License)	Free	
6	Oscilloscope	Used to observe PWM waveforms and confirm correct signal duty cycles for verifying motor speed and performance.	Lab Equipment Software	Institution	

5.4 Software Environment Setup

The software environment for the RoboRide project was built using Ubuntu Linux as the primary development platform and the Raspberry Pi Zero W as the hardware controller. Ubuntu was selected for its stability, open-source nature, and strong compatibility with embedded system tools.

The C programming language was used to develop the robot's control logic, compiled using the GCC compiler. The WiringPi and softPwm libraries were integrated to control the GPIO pins and manage Pulse Width Modulation (PWM) for motor speed regulation.

The Raspberry Pi OS Lite was installed on the Raspberry Pi Zero W, allowing headless operation via SSH over Wi-Fi. This setup ensured smooth communication between the development laptop and the robot, enabling quick debugging and real-time code execution.

5.5 Setup Procedure

The setup process for the RoboRide project involved configuring both the Ubuntu development system and the Raspberry Pi Zero W for motor control implementation.

1. Ubuntu Installation
 - Created a bootable USB using *Rufus* and installed Ubuntu 22.04 LTS for a stable, open-source environment.
 - Updated system packages and installed essential tools:
 - `sudo apt update && sudo apt upgrade -y`
 - `sudo apt install build-essential git -y`
2. Raspberry Pi OS Setup
 - Used Raspberry Pi Imager to flash Raspberry Pi OS Lite onto a microSD card.
 - Configured Wi-Fi credentials and enabled SSH in the Imager settings for headless access.
 - Inserted the microSD card into the Raspberry Pi Zero W and powered it using a 5V source.
 - Verified connection:
 - `ping raspberrypi.local`
3. Library and Tool Installation
 - Installed WiringPi for GPIO and PWM control:
 - `sudo apt install wiringpi -y`
 - `gpio -v`
 - Used softPwm functions for speed modulation through software-based PWM.
4. Code Development and Deployment
 - Wrote and edited the control code in C using the nano editor on Ubuntu.
 - Compiled and executed the program:
 - `gcc RoboRide.c -lwiringPi -o RoboRide`
 - `./RoboRide`
5. Circuit Verification
 - Designed and verified the wiring using Fritzing.
 - Conducted basic motor tests using terminal commands to confirm proper GPIO control.

6. Software Requirements Specification (SRS)

Project:

Document: Software Requirements Specification

Author: Bilal Hussain

Date: October 12, 2025

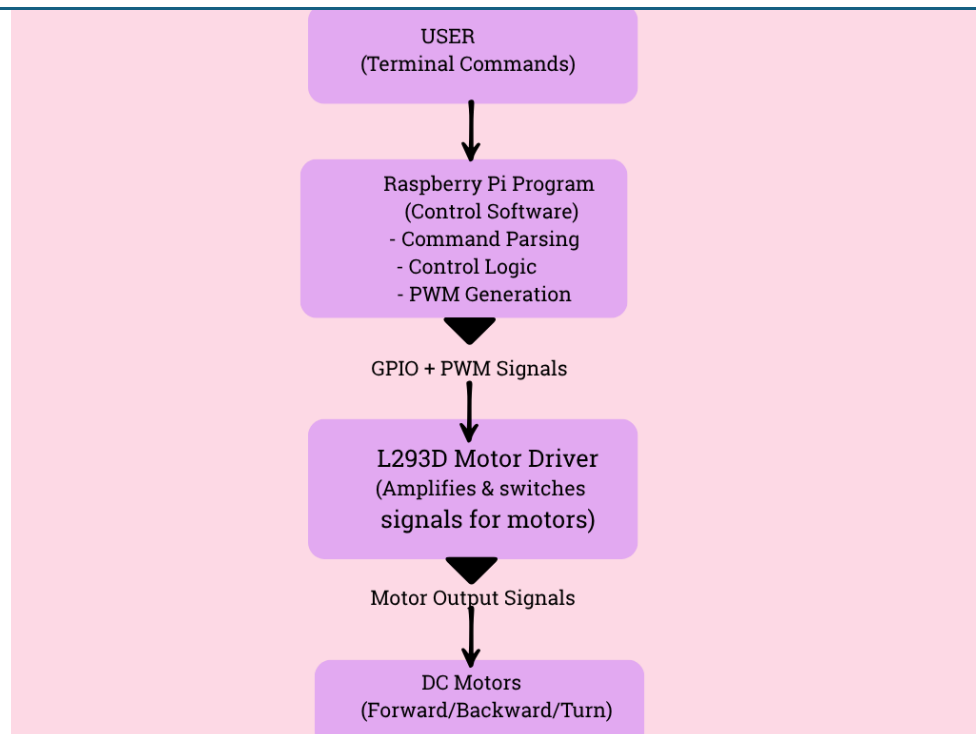
Version: 1

Overall Description

Background: There is a need for a simple and low-cost program that demonstrates how software can control hardware in real time for educational robotics and motor control applications.

Project Overview: This program allows the Raspberry Pi to control a two –wheel robot by reading user commands and using GPIO and PWM signals to move the motors in different directions and speeds.

Block Diagram:



Overall Description

Block Descriptions:	USER (Terminal Commands): The user enters control commands such as forward, backward, left, right, stop, or speed settings through the terminal. These commands act as the input to the system. Raspberry Pi Program (Control Software): This software reads the user's commands, parses them, decides the correct action, and generates the appropriate GPIO and PWM signals. It handles command interpretation, control logic, and motor speed/direction control. L293D Motor Driver: The motor driver receives low-power control signals (GPIO and PWM) from the Raspberry Pi and amplifies them to the required voltage and current levels. It safely drives the DC motors using an H- bridge configuration. DC Motors: The DC motors convert the electrical signals from the motor driver into physical movement. They perform the robot's motion actions such as moving forward, backward, turning, or stopping.
Typical User:	The typical user is a student, or beginner in electronics or embedded systems who wants to learn how software can control hardware. They may have basic programming or Raspberry Pi knowledge but do not need advanced robotics experience. The system is designed so the user can operate the robot using simple terminal commands while observing real- time motion feedback.
Constraints:	• The program must run on a Raspberry Pi with limited GPIO pins and processing power. • The software depends on the Wiring Pi and soft PWM libraries, which must be installed and supported by the OS. • The robot can only be controlled through a terminal (no GUI or wireless control in the current version). • Power limitations of AA batteries and the motor driver may affect performance and runtime. • Physical wiring space is limited, making hardware connections sensitive to errors or loose contacts. • The system must stop motors before changing direction to prevent hardware damage.

Specific Requirements	
External Interfaces:	1) User Input Interface: Command-line terminal where the user types of commands (FWD, BCK, LFT, RGT, STOP,1,2, EXIT). 2) Hardware Output Interface: GPIO pins on the Raspberry Pi send digital signals and PWM signals to the L293D motor driver to control motor direction and speed. 3) Display/Feedback Interface: Text messages shown in the terminal to confirm actions (e.g., "Moving Forward," "Speed set to Fast", "Invalid command").
Specific Requirements:	Below listed are all requirements that exist for this program and given each a MuSCoW rating.

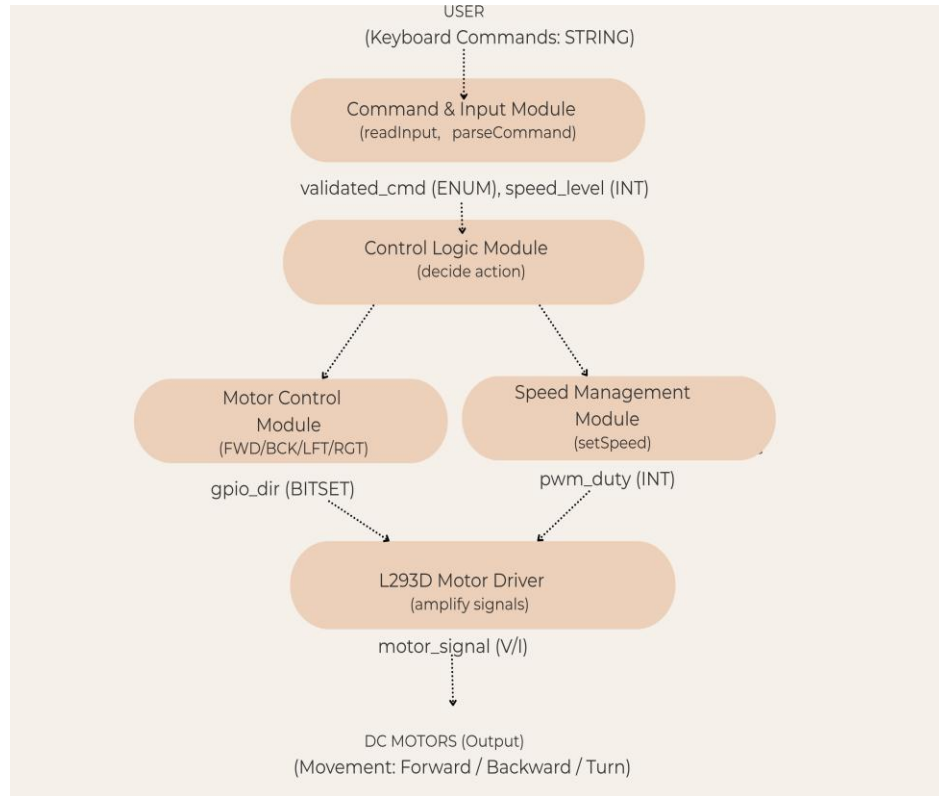
Number	Requirement	MuSCoW
1	platform requirements (PC, MAC, web-based, etc.)	Must
2	interface requirements (command line, interactive text, GUI, etc.)	Must
3	user I/P requirements (acceptable ranges for user-specified values, and defaults where applicable)	Must
4	calculation requirements (accuracy, precision of calculations ,and random numbers, etc.)	Should
5	visual O/P requirements (precision of displayed values, layout of tables, etc.)	Should
6	storage/control O/P requirements (local/cloud storage, devices to be controlled, etc.)	Could
7	documentation provided (manuals, instructions, etc.)	Must

7. Software Design Description (SDD)

Project:	
Document:	Software Design Document
Author:	Nikunj Patel
Date:	October 11, 2025
Version:	1

System Description	
System Description:	The software controls a two-wheel robot using a Raspberry Pi. It reads user commands from the terminal and translates them into motor actions such as forward, backward, turning, and stopping. The software uses GPIO and PWM signals to set motor direction and speed safely and reliably.
Development Platform:	The program is developed on Raspberry Pi OS using the GCC compiler and C programming language. The Wiring Pi and softPwm libraries are used for GPIO control and PWM generation. A computer or the Raspberry Pi itself is used to write and compile the code.
Execution Platform:	The program runs directly on the Raspberry Pi Zero W (or any compatible Raspberry Pi). The Pi sends control signals to the L293D motor driver, which drives the DC motors of the robot.

Data Flow Diagram:



System Interfaces:

Input (Terminal prompt)

```
Robot Motor Controller
Commands:
FWD  = forward
BCK  = backward
LFT  = spin left
RGT  = spin right
STOP = stop
1    = set slow speed (70%)
2    = set fast speed (90%)
EXIT = quit program

Enter command: |
```

Normal output (status/feedback)

```
Enter command: BCK
Moving backward at 90% duty

Enter command: LFT

Spinning left at 90% duty

Enter command: FWD
Moving forward at 90% duty
```

Error messages (invalid/unsafe input)

```
Robot Motor Controller
Commands:
FWD  = forward
BCK  = backward
LFT  = spin left
RGT  = spin right
STOP = stop
1    = set slow speed (70%)
2    = set fast speed (90%)
EXIT = quit program

Enter command: yigjjjjhkhvhvhhbvccmgjvcvjgv
Invalid command!

Enter command:
```

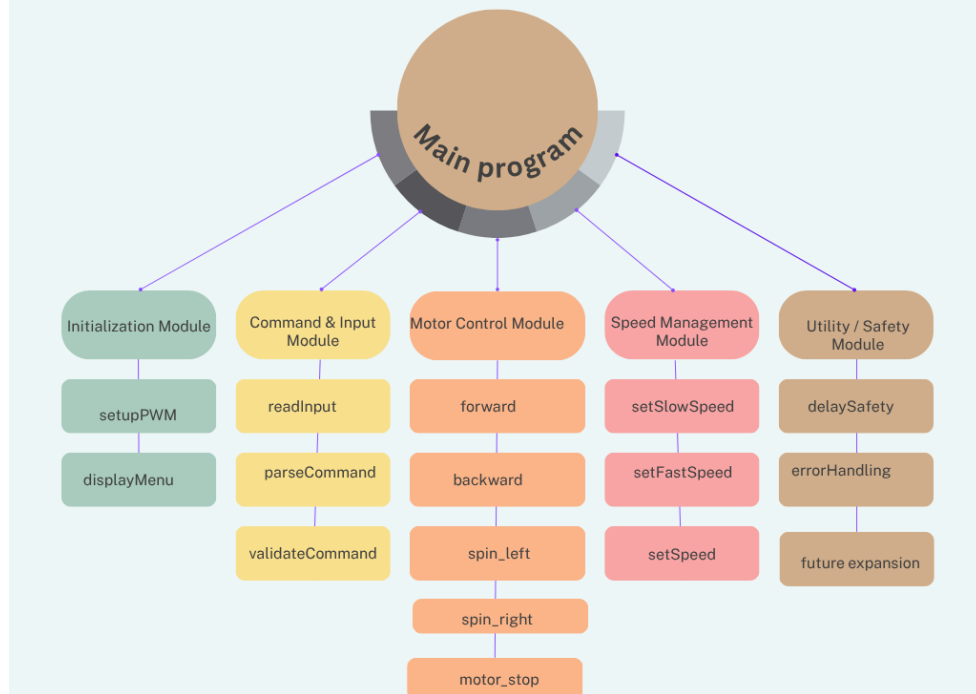
Program exit

```
Enter command: stop
Invalid command!

Enter command: EXIT
Exiting program...
Segmentation fault
```

Detailed Design

Hierarchy Diagram:



Modules (one for each block shown in the hierarchy diagram)

Module Name: Initialize System

Purpose: Prepare hardware and software environment (set GPIO modes, start softPWM show menu).

Input: None

Output: Initialized GPIO pins, active PWM channels, on- screen command menu.

Module Name: Read and parse Command

Purpose: Read user input from terminal, validate it, and translate to an internal action+ optional speed value.

Input: User text (e.g., FWD, BCK, LFT, RGT, STOP, 1, 2, EXIT).

Output: validated_command (enum/action) and speed_level (int when applicable).

Module Name: Execute Motion Command

Purpose: Drive direction pins and PWM enables to move the robot. (Forward /backward /spin) with safe stop before reversals.)

Input: direction (enum:FWD/BCK/LFT/RGT/STOP), current speed (0-100)

Output: GPIO signals for IN1-IN4, PWM duty updates on ENA/ENB; physical motion.

Module Name: Set Motor Speed

Purpose: Map speed choices to PWM duty cycles and apply them to enable pins.

Input: speed_level (e.g., SLOW/FAST) or numeric speed (0-100)

Output: PWM duty cycle applied to ENA/ENB: confirmation text (e.g., “speed set to FAST (90%).”

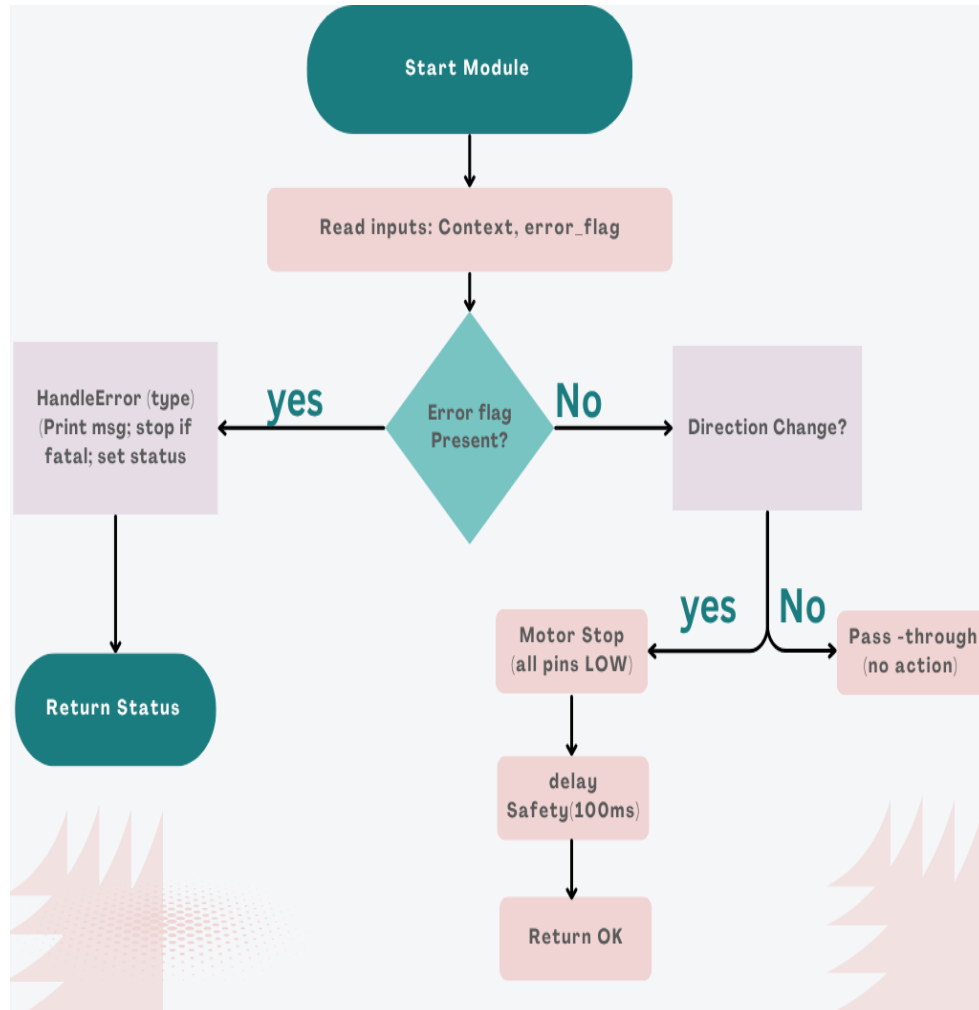
Module Name: Enforce Safety & Handle Errors

Purpose: Insert safety delays, stop motors before direction changes and handle invalid states gracefully.

Input: Requests from other modules (e.g., “prepare to reverse”), error flags.

Output: motor_stop() action delaysafety , timing, clear error/status messages.

Flow chart:



RoboRide Report – Bilal Hussain – Nikunj Patel

Test Values	
Reliability Test:	(Valid inputs: low, mid, high – and expected output) • Input: FWD at slow speed (70%) Expected output: Robot moves forward smoothly. • Input: Fwd. at fast speed (90%) Expected Output: Robot moves forward faster without errors. • Input: Run commands repeatedly for 30 minutes Expected Output: Program continues to work without crashing or freezing.
Robustness Test:	• Input: XYZ Expected Output: “Invalid command!” message and no movement. • Input: Empty input (press Enter) Expected output: Program ignores or shows error without crashing. • Input: Mixed case (e.g., fWd) Expected Output: Command still works (case- insensitive) or shows clear error.
Exceptions Test:	▪ Exception: Lost PWM or GPIO initialization failure. ▪ Handling: Program prints error and exits safely. ▪ Exception: Program is closed while motors are running. ▪ Handling: motor_stop() is called to stop all motors before exit. ▪ Exception: User enters command rapidly (e.g., FWD then BCK quickly) ▪ Handling: Program always stops motors before changing direction to prevent hardware damage. ▪ Exception: Speed not set (null or Invalid) ▪ Handling: Default speed (slow) is used and error message shown.

8. Circuit Design

8.1 Circuit Diagram

- The complete circuit for the *RoboRide* system was designed and simulated using Fritzing before physical implementation. The diagram below illustrates how all hardware components are interconnected. (fig 1)

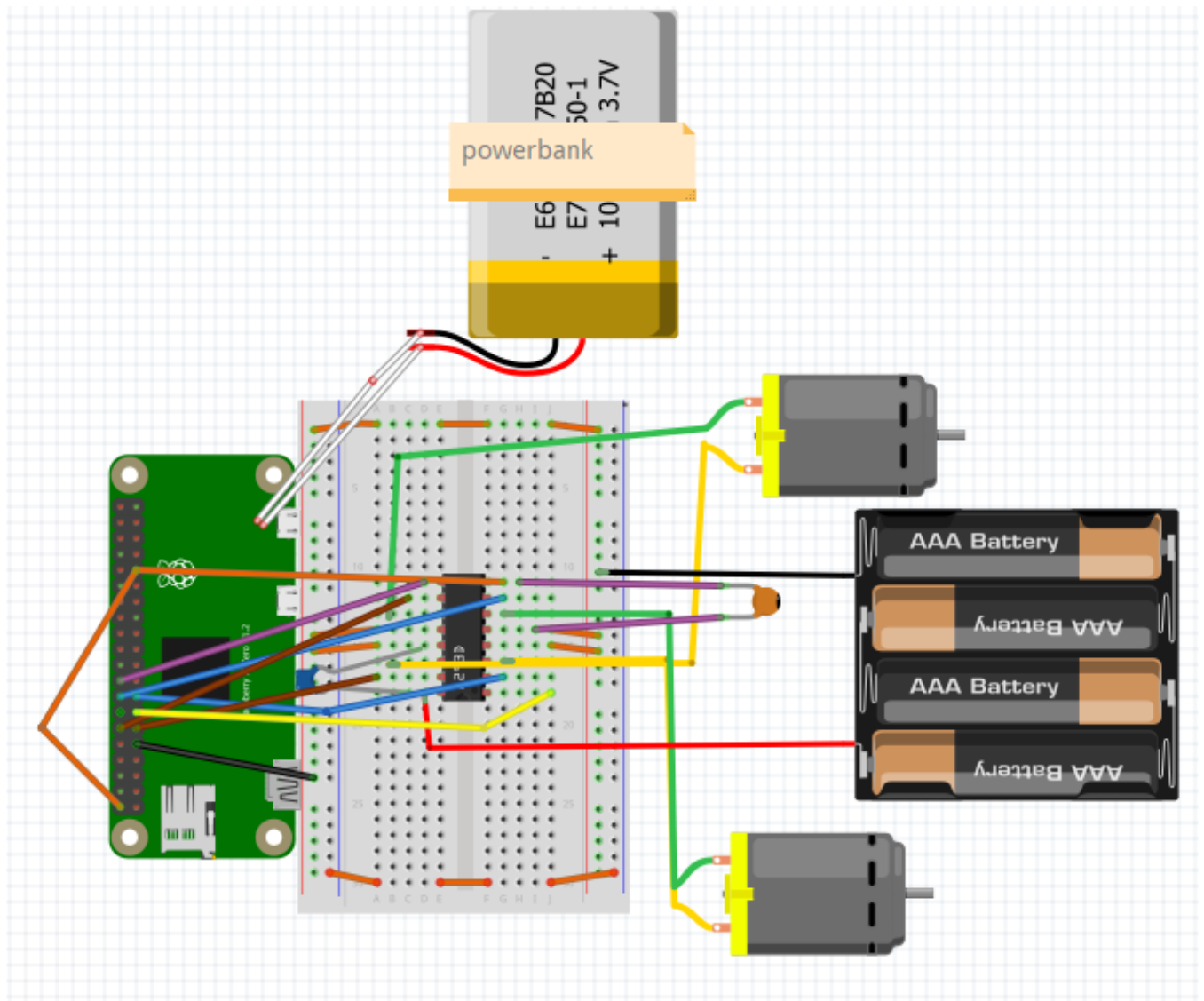


Figure 1 : Circuit diagram showing Raspberry Pi, L293D motor driver, DC motors, and dual power configuration.

8.3 Imagery

Below attached image illustrates the actual physical wiring.(fig 3)

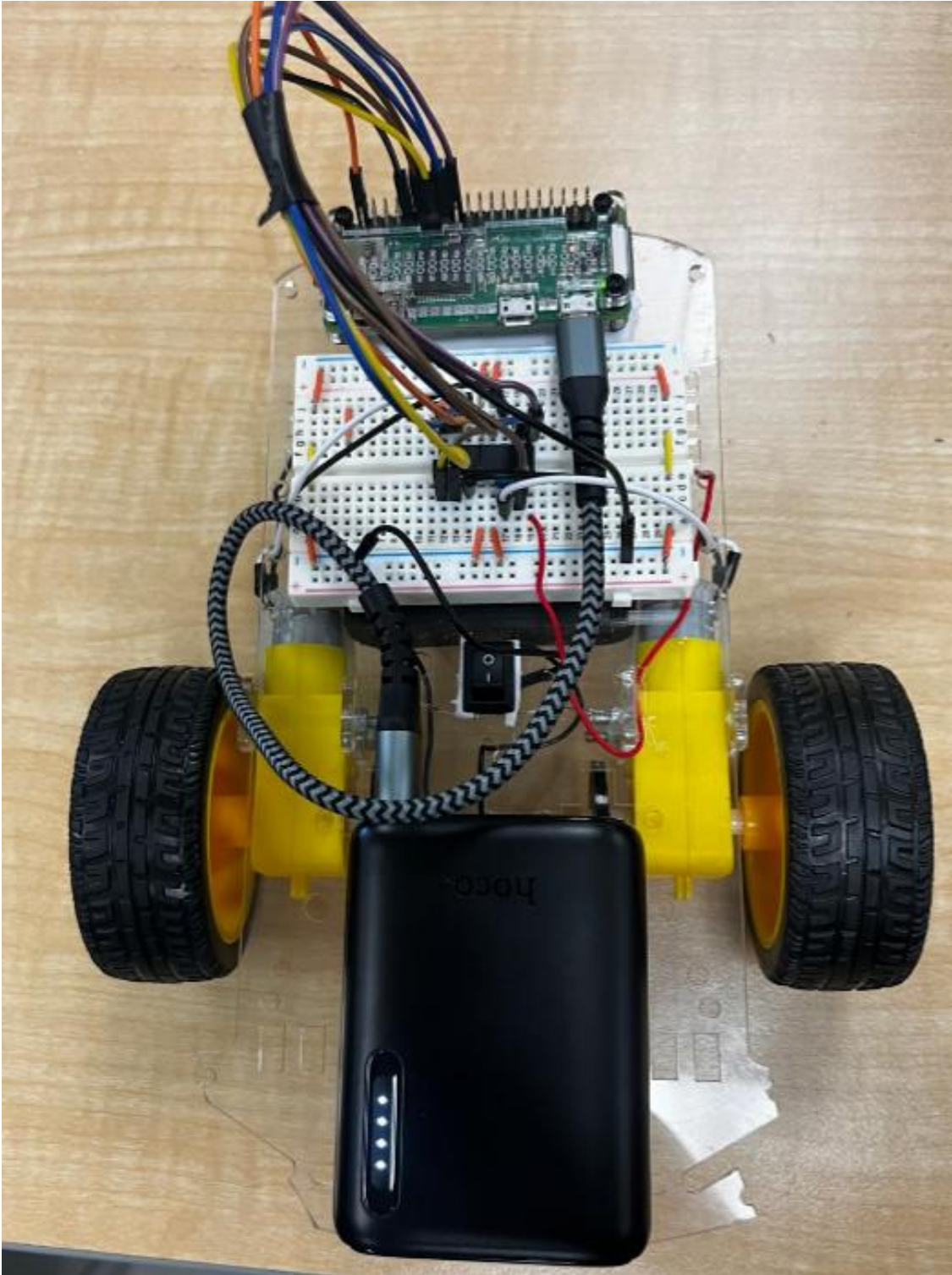


Figure 3: Image from our robot for better understanding.

8.4 Wiring Description

- The following table lists all wiring connections between the Raspberry Pi, L293D motor driver, power supply, and motors.

No.	Component / Pin	Connected To	Component Pin	Description
1	Raspberry Pi GPIO 17 (WiringPi 0)	L293D Pin 2 (IN1)	Digital Output	Controls Motor A direction 1.
2	Raspberry Pi GPIO 18 (WiringPi 1)	L293D Pin 7 (IN2)	Digital Output	Controls Motor A direction 2.
3	Raspberry Pi GPIO 27 (WiringPi 2)	L293D Pin 1 (ENA)	PWM Output	Enables Motor A speed control via PWM.
4	Raspberry Pi GPIO 22 (WiringPi 3)	L293D Pin 10 (IN3)	Digital Output	Controls Motor B direction 1.
5	Raspberry Pi GPIO 23 (WiringPi 4)	L293D Pin 15 (IN4)	Digital Output	Controls Motor B direction 2.
6	Raspberry Pi GPIO 24 (WiringPi 5)	L293D Pin 9 (ENB)	PWM Output	Enables Motor B speed control via PWM.
7	L293D Pin 3 & 6	DC Motor A	Motor Output	Connects to left motor terminals.
8	L293D Pin 11 & 14	DC Motor B	Motor Output	Connects to right motor terminals.
9	L293D Pin 8 (Vcc2)	4 × AA Battery Pack ($\approx 6\text{ V} +$)	Power Input	Provides motor power supply.
10	L293D Pin 16 (Vcc1)	Raspberry Pi 5 V (Pin 2 or 4)	Logic Power	Supplies logic voltage to driver IC.
11	L293D Ground Pins (4, 5, 12, 13)	Common Ground (Pi GND + Battery $-$)	Ground	Completes circuit return path.
12	Raspberry Pi GND (Pin 6 or 9)	Breadboard Ground Rail	Ground	Shared ground for stable reference.
13	Breadboard Power Rail	Power Bank 5 V Output	Power	Provides regulated 5 V supply to Pi.

9. Implementation

9.1 Code Structure

The *RoboRide* program was written in C language and executed on a Raspberry Pi using the WiringPi and softPwm libraries. The overall structure of the code follows a modular and logical flow that enables clear control of motor direction and speed. The major sections of the code are outlined below:

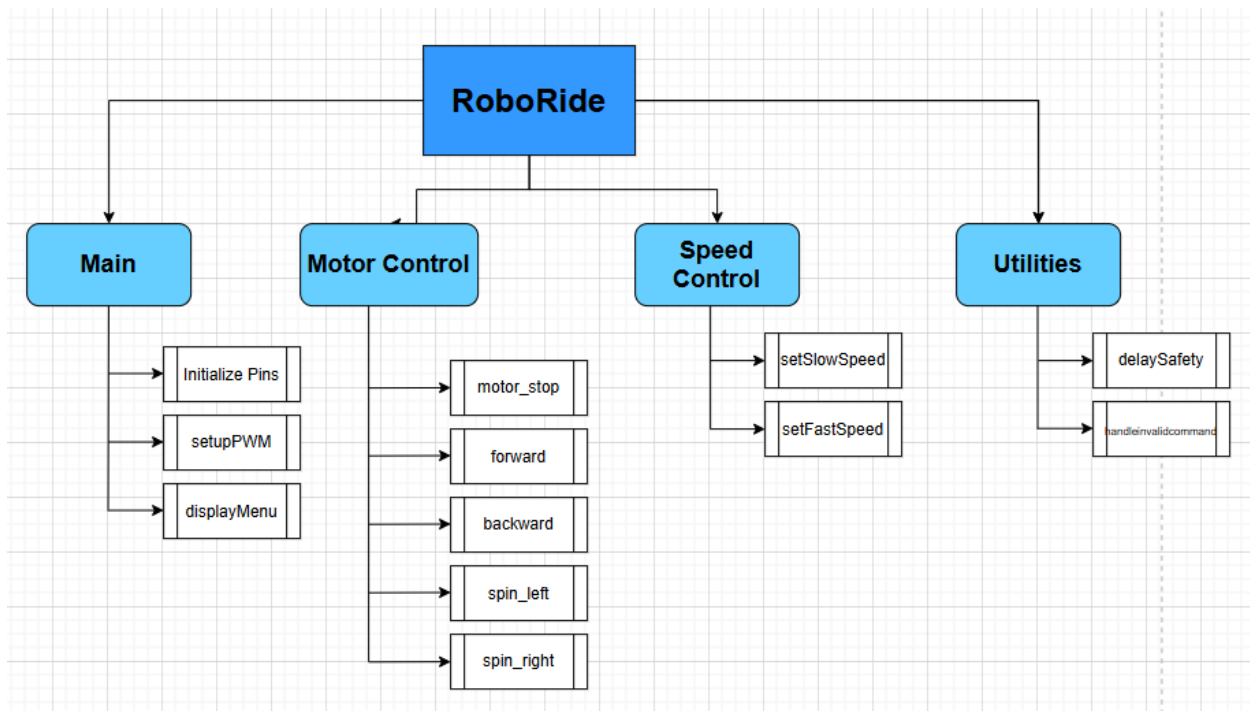


Figure 4: Code Structure Diagram of the RoboRide System created using Draw.io. (draw.io, n.d.)

9.2 Explanation of code for better understanding

- **Header Inclusions:**

The program begins with the inclusion of the necessary libraries — `wiringPi.h`, `softPwm.h`, `stdio.h`, and `string.h`. These enable GPIO control, PWM generation, and command input handling.
- **Pin Mapping Definitions:**

Each Raspberry Pi pin is defined using `#define` statements, clearly identifying the control pins for Motor A and Motor B, their direction inputs (IN1–IN4), and enable pins (ENA and ENB). This improves readability and allows easy reconfiguration.
- **Speed Constants:**

The code defines two speed presets, `SLOW_SPEED` (70%) and `FAST_SPEED` (90%), enabling controlled operation at different speeds using PWM.
- **Motor Control Functions:**

Core movement functions are implemented to handle motion in all directions:

 - `motor_stop()` – stops both motors safely by setting all control pins LOW and PWM to 0.
 - `forward(speed)` – moves both motors forward at a given PWM duty cycle.
 - `backward(speed)` – reverses both motors.
 - `spin_left(speed)` and `spin_right(speed)` – control opposite wheel directions to rotate the car on its axis.
- **Main Function Logic:**

The `main()` function initializes the GPIO pins using `wiringPiSetup()`, configures them as outputs, and creates PWM channels for both motors using `softPwmCreate()`. It then enters a command-driven loop where user input (via terminal) determines the robot's action. Valid commands such as "FWD", "BCK", "LFT", "RGT", "STOP", "1", "2", and "EXIT" control motion, speed, or program termination.
- **Input Handling and Safety:**

The code uses `strcmp()` for reliable string comparison and includes safety delays when changing directions to protect the motors and driver IC. It also ensures all motion stops before executing a new direction command.
- **Code Organization and Readability:**

Functions are modular and self-contained, with meaningful variable names and consistent commenting for each section. This makes the code easily maintainable and extendable for future integration of sensors or wireless modules.

9.3 Key Code Snippets

- **Motor Stop Function**

```
// Stop both motors completely
void motor_stop() {
    digitalWrite(IN1, LOW);
    digitalWrite(IN2, LOW);
    digitalWrite(IN3, LOW);
    digitalWrite(IN4, LOW);
    softPwmWrite(ENA, 0);
    softPwmWrite(ENB, 0);
    delay(100); // short delay before direction change (safety)
}
```

Figure 5: Code Snippet for Motor Stop Function

Explanation:

This function stops both motors safely by setting all input pins to LOW and PWM duty cycles to zero. A short delay prevents motor driver damage when switching direction.

- **Forward Movement**

```
// Move the robot forward
void forward(int speed) {
    motor_stop(); // ensure motors are off before changing direction
    digitalWrite(IN1, HIGH);
    digitalWrite(IN2, LOW);
    digitalWrite(IN3, HIGH);
    digitalWrite(IN4, LOW);
    softPwmWrite(ENA, speed);
    softPwmWrite(ENB, speed);
}
```

Figure 6: Code snippet for motor forward function

Explanation:

This function drives both motors forward at a given PWM speed (between 0–100). The `motor_stop()` call ensures smooth transitions and avoids polarity conflict.

- **Main Control Loop**

```
// Main control loop
while (running) {
    printf("\nEnter command: ");
    scanf("%s", cmd);

    if (strcmp(cmd, "FWD") == 0) {
        forward(speed);
        printf("Moving forward at %d%% speed.\n", speed);
    }
    else if (strcmp(cmd, "BCK") == 0) {
        backward(speed);
        printf("Moving backward at %d%% speed.\n", speed);
    }
    else if (strcmp(cmd, "LFT") == 0) {
        spin_left(speed);
        printf("Spinning left at %d%% speed.\n", speed);
    }
    else if (strcmp(cmd, "RGT") == 0) {
        spin_right(speed);
        printf("Spinning right at %d%% speed.\n", speed);
    }
    else if (strcmp(cmd, "STOP") == 0) {
        motor_stop();
        printf("Motors stopped.\n");
    }
    else if (strcmp(cmd, "1") == 0) {
        speed = SLOW_SPEED;
        printf("Speed set to SLOW (%d%%)\n", speed);
    }
    else if (strcmp(cmd, "2") == 0) {
        speed = FAST_SPEED;
        printf("Speed set to FAST (%d%%)\n", speed);
    }
    else if (strcmp(cmd, "EXIT") == 0) {
        motor_stop();
        running = 0;
        printf("Exiting program... Goodbye!\n");
    }
    else {
        printf("Invalid command! Please try again.\n");
    }
}
```

Figure 7: Code snippet for Main Control Loop

Explanation:

The program continuously listens for user commands and executes the corresponding motor actions. This structure simulates a real-time control interface for testing motion functions interactively.

9.4 Challenges and Solutions

Challenge	Description	Solution Implemented
Motor startup hesitation	Motors sometimes required a small manual push to start due to low initial torque.	Adjusted PWM values (70% for slow, 90% for fast) and ensured both motors stop before direction changes.
Pin congestion on Raspberry Pi	Limited GPIO space and tight physical wiring caused confusion during setup.	Created a clear pin map and verified each connection with a multimeter and Fritzing schematic.
Polarity issues	Robot initially moved in reverse when “forward” was commanded.	Swapped the polarity of one motor’s connection on the L293D output pins.
Connection to Pi	Loose jumper wires occasionally caused motor flickering.	Used tighter male-female jumper connectors and secured the breadboard connections.
Pi Connection	Had problems like pi was not connecting to hotspot.	Enabled 2.4Ghz hotspot connection.

10. Testing and Results

10.1 Testing Plan

- **T1 – Forward Movement Test**

This test verified the robot's ability to move straight ahead when the "FWD" command was entered. Both DC motors rotated in the forward direction, powered through the L293D motor driver. The robot maintained a straight trajectory with minimal deviation, confirming correct motor wiring and signal output from the Raspberry Pi. (fig 8)

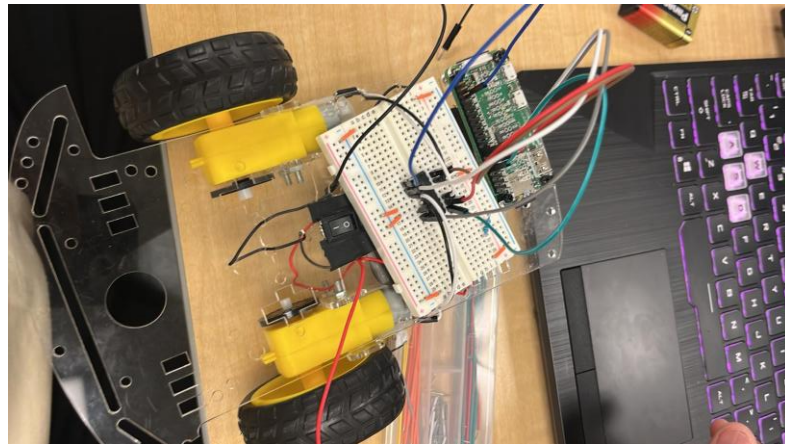


Figure 8: Forward Movement Test. Video link : [Bilal2866/ENGG-PROJECT-3](https://www.youtube.com/watch?v=Bilal2866/ENGG-PROJECT-3) at Week-5-Phase-1

- **T2 – Backward Movement Test**

The purpose of this test was to validate the reverse functionality. When the "BCK" command was given, both motors reversed direction. The motion was smooth and stable, confirming proper polarity configuration and successful execution of reverse logic in the program. (fig 9)

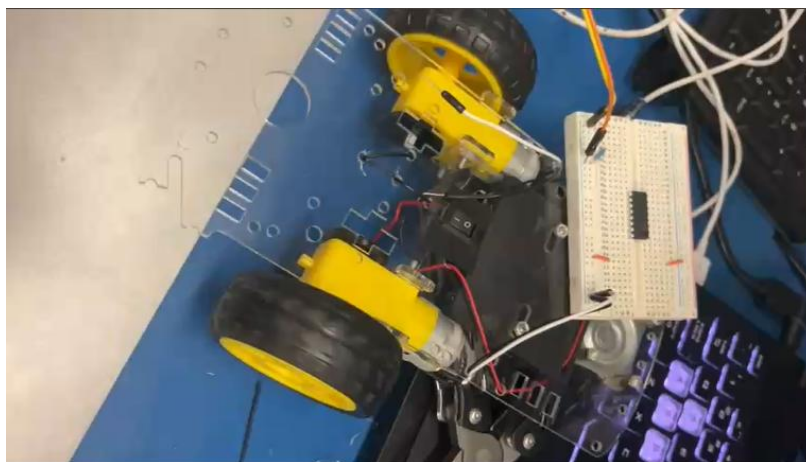


Figure 9: Backward Movement Test. Video link : [Bilal2866/ENGG-PROJECT-3](https://www.youtube.com/watch?v=Bilal2866/ENGG-PROJECT-3) at Week-5-Phase-1

- **T3 – Left Turn Test**

This test ensured that the robot could rotate left on its axis. The right motor rotated forward while the left motor rotated backward. The robot performed an in-place left turn, demonstrating accurate direction control through the L293D's differential drive system. (fig 10)

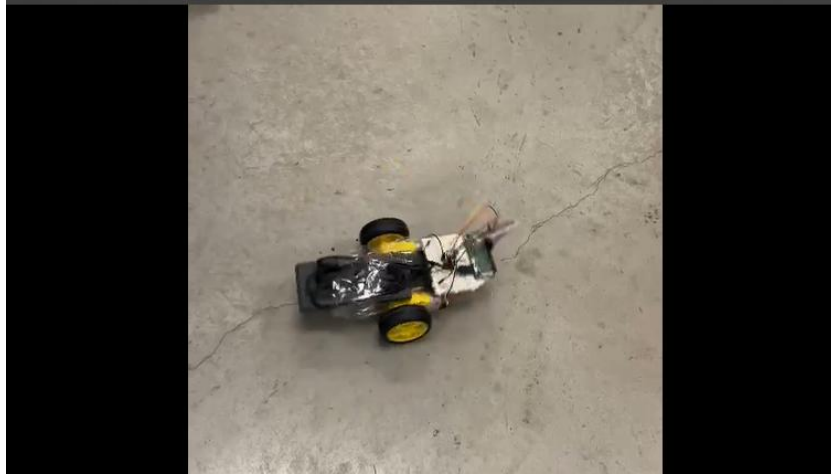


Figure 10: Left Turn Test. [Video link : Bilal2866/ENGG-PROJECT-3 at Week-5-Phase-1](https://www.youtube.com/watch?v=Bilal2866/ENGG-PROJECT-3)

- **T4 – Right Turn Test**

The “RGT” command was tested for correct right rotation. The left motor rotated forward, and the right motor rotated backward, producing a clean spin to the right. The motion confirmed correct GPIO mapping and direction control signals. (fig 11)

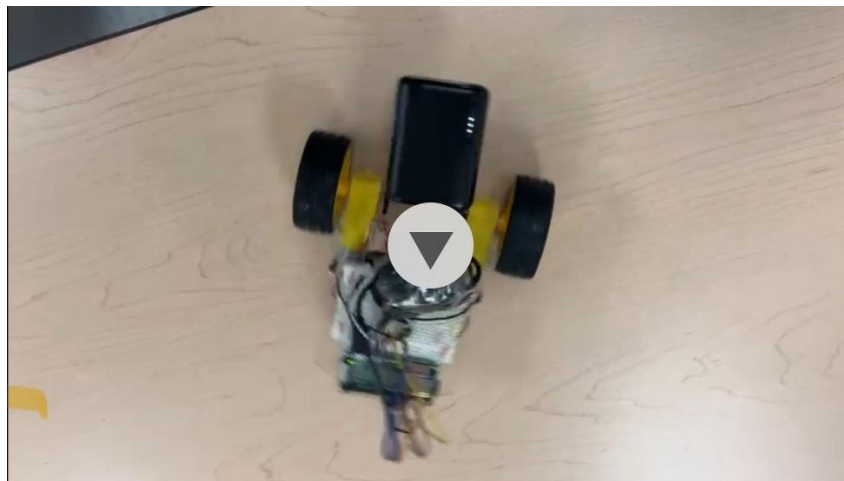


Figure 11: Backward Movement Test. [Video link : Bilal2866/ENGG-PROJECT-3 at Week-5-Phase-1](https://www.youtube.com/watch?v=Bilal2866/ENGG-PROJECT-3)

- **T5 – Speed control test through duty cycle**

Tested PWM-based speed control at 70% (slow) and 90% (fast) duty cycles. Oscilloscope confirmed clean PWM signals with accurate duty variation. Robot speed changed smoothly between modes, verifying effective speed modulation. (fig 12)



Figure 12: Duty Cycle Test. **Video link :** [Bilal2866/ENGG-PROJECT-3 at Week-5-Phase-1](https://www.youtube.com/watch?v=Bilal2866/ENGG-PROJECT-3 at Week-5-Phase-1)

- **T6 – Oscilloscope PWM Test**

An oscilloscope was connected to the ENA and ENB pins to observe PWM waveforms. Clean square waves were displayed with duty cycles accurately reflecting the programmed values. The test verified that PWM signals were stable, consistent, and free of electrical noise. (fig 13)

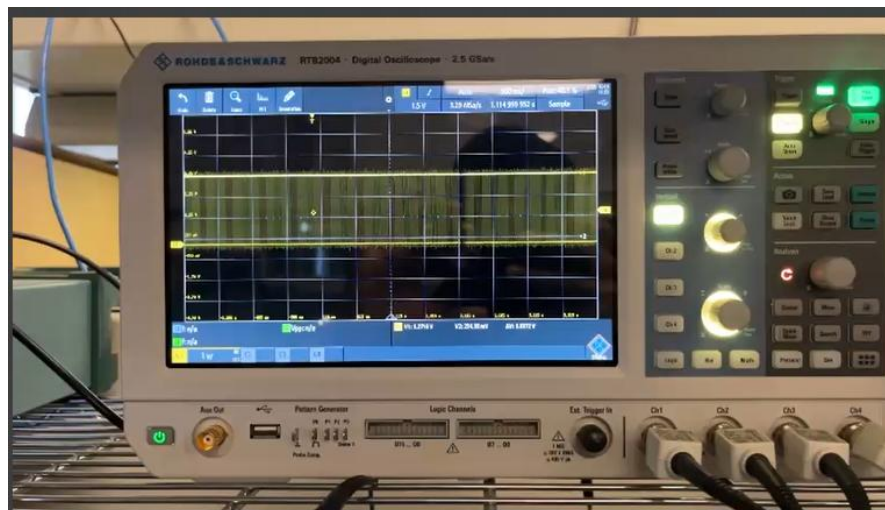


Figure 13: Oscilloscope PWM Test. **Video link :** [Bilal2866/ENGG-PROJECT-3 at Week-5-Phase-1](https://www.youtube.com/watch?v=Bilal2866/ENGG-PROJECT-3 at Week-5-Phase-1)

- **T7 – Motor Stop Test**

This test checked the response to the “STOP” command. Both motors halted immediately upon receiving the command, with PWM duty reduced to zero. This verified that the braking mechanism and safety delay functions operated as intended. (fig 14)



Figure 14: Motor Stop Test. **Video link :** [Bilal2866/ENGG-PROJECT-3 at Week-5-Phase-1](https://github.com/Bilal2866/ENGG-PROJECT-3-at-Week-5-Phase-1)

- **T8 – Power Interruption Test**

In this test, the system was powered off mid-operation and then restarted. Upon reboot, the robot resumed proper function without configuration loss. The result demonstrated reliable hardware initialization and stable software re-execution after power cycling.

Note : All Test videos can be found in GitHub. Please follow the link.
[Bilal2866/ENGG-PROJECT-3 at Week-5-Phase-1](https://github.com/Bilal2866/ENGG-PROJECT-3-at-Week-5-Phase-1)

10.2 Test Results

The testing phase focused on verifying both the software control logic and the hardware performance of the RoboRide system. The objectives were to ensure accurate motor responses to directional commands, validate PWM signal integrity using an oscilloscope, and confirm system stability under different speed conditions. Each test was conducted on the assembled prototype powered by the Raspberry Pi Zero W and L293D motor driver.

Test Case ID	Test Description	Expected Result	Actual Result	Status
T1	Verify forward movement of both motors when “FWD” command is entered.	Both motors rotate forward; robot moves straight ahead.	Robot moves straight with minimal deviation.	
T2	Verify backward movement using “BCK” command.	Both motors rotate in reverse; robot moves backward smoothly.	Functioning as expected.	
T3	Test left rotation using “LFT” command.	Right motor rotates forward; left motor rotates backward, causing left spin.	Achieved correct turning motion.	
T4	Test right rotation using “RGT” command.	Left motor rotates forward; right motor rotates backward, causing right spin.	Correct rotation observed.	
T5	Test speed control between slow (70%) and fast (90%) settings.	Motor speed adjusts smoothly with visible difference between modes.	Smooth transition observed.	
T6	Oscilloscope PWM test for ENA and ENB pins.	PWM signal frequency and duty cycle correspond to set speed percentages.	Verified clean square wave with accurate duty modulation on oscilloscope.	
T7	Test emergency stop (“STOP” command).	Motors stop immediately; no further motion.	Works as expected.	
T8	Power interruption test.	Robot resumes correct operation after power restoration.	Successfully resumes after restart.	

10.3 Observations

- The robot successfully executed all motion commands — forward, backward, left, and right rotations — without wiring or logic errors.
- PWM output observed on the oscilloscope showed clean, stable square waves on both ENA and ENB pins.
- Duty cycle variation between slow (~70%) and fast (~90%) modes was consistent with programmed values.
- Motor response time was quick, with smooth acceleration and deceleration between speed modes.
- Minor delay at startup was observed due to motor inertia but resolved once movement began.
- The STOP command halted both motors immediately, confirming reliable braking control.
- Power delivery from the 5000 mAh power bank and 6 V battery pack was stable, showing no voltage drop during motion.
- No overheating or short-circuit issues occurred during extended operation.
- The system maintained consistent performance after power interruptions and restarts.
- Overall, RoboRide demonstrated high reliability, precision in speed control, and smooth maneuverability.

11. Discussion

11.1 Analysis of Results

- The *RoboRide* project achieved its main objectives of developing a compact, controllable robotic car powered by a Raspberry Pi and driven through an L293D motor driver. The robot successfully demonstrated forward, backward, left, and right motion at two selectable speed levels, controlled through command-line inputs. The PWM-based speed control and the logic wiring between the Pi and motor driver functioned reliably, validating both the circuit design and software logic.
- Minor issues such as motor start hesitation and occasional signal lag were observed, but they did not significantly impact the system's overall functionality. The final tests confirmed that the project met its design goals for motion control, power management, and system stability, making it a solid proof of concept for short-range robotic mobility.

11.2 Limitations

Despite meeting its objectives, several limitations were identified during testing and operation:

- **Limited torque and load capacity** — The DC motors are suitable only for light payloads.
- **Short operating time** — The power bank and AA battery pack require frequent recharging or replacement during extended sessions.
- **No obstacle detection** — The system currently relies entirely on manual input with no sensors for autonomous navigation.
- **Physical wiring congestion** — The Raspberry Pi's compact GPIO layout makes connections crowded and prone to disconnection.
- **Flat-surface dependency** — The chassis and wheels perform best on smooth, indoor surfaces, limiting terrain adaptability.

11.3 Potential Improvements

To enhance the *RoboRide* system's performance and real-world applicability, the following improvements are recommended:

- **Add sensors** such as ultrasonic or infrared modules for obstacle avoidance and autonomous control.
- **Integrate wireless connectivity** (e.g., Wi-Fi or Bluetooth) for mobile app-based remote control.
- **Include a camera module** for live video streaming/recording or object recognition tasks.
- **Optimize power distribution** by integrating a single, rechargeable Li-ion battery with voltage regulation for both logic and motors.
- **Develop a user interface** with visual feedback for direction, speed, and system status.
- **Refine the chassis** with a sturdier frame and better rubberized wheels to improve traction and load handling.

12. Conclusion

- The *RoboRide* project successfully demonstrated the design, wiring, and programming of a Raspberry Pi-based robotic car capable of controlled motion in multiple directions. Through the integration of the L293D motor driver, DC motors, and dual power sources, the team achieved reliable bidirectional movement and speed control using PWM signals. The project provided a comprehensive understanding of motor interfacing, circuit design, and software-hardware synchronization.
- During development, key challenges such as wiring congestion, polarity confusion, and motor startup hesitation were systematically analyzed and resolved through careful testing and optimization. These experiences strengthened problem-solving, debugging, and teamwork skills while reinforcing fundamental concepts in embedded systems and electronics.
- From a learning perspective, *RoboRide* served as an excellent introduction to practical robotics, showcasing how theoretical principles—such as GPIO control, pulse-width modulation, and current amplification—translate into functional real-world systems. The prototype also highlights potential industrial and educational applications, including automated material handling, indoor delivery systems, and STEM learning platforms.

Overall, the project not only met its objectives but also laid the foundation for future enhancements such as obstacle detection, wireless control, and autonomous navigation, moving closer to the development of a fully intelligent mobile robot.

13. References

- List of all resources used, including tutorials, datasheets, libraries, and textbooks.
- Raspberry Pi Foundation. (2021). *Raspberry Pi 4 Model B technical datasheet* [PDF]. Raspberry Pi Foundation. Retrieved from <https://datasheets.raspberrypi.com/rpi4/raspberry-pi-4-datasheet.pdf>
- STMicroelectronics. (2003). *L293D: Push-Pull Four Channel Driver with Diodes* [PDF]. STMicroelectronics. Retrieved from <https://www.st.com/resource/en/datasheet/l293d.pdf>
- Digi-Key Electronics. (n.d.). *L293D – Bipolar Motor Driver – STMicroelectronics*. Digi-Key Electronics. Retrieved from <https://www.digikey.com/en/products/detail/stmicroelectronics/L293D/634700>
- Panasonic Corporation. (2020). *AA Alkaline Battery (LR6) datasheet* [PDF]. Panasonic Industry. Retrieved from <https://industry.panasonic.com/downloads/AA-LR6-battery-datasheet.pdf>
- Adafruit Industries. (2016). *DC Hobby Gearmotor – 6V, 200 RPM datasheet* [PDF]. Adafruit. Retrieved from https://cdn-shop.adafruit.com/product-files/711/711_datasheet.pdf
- SanDisk. (2021). *SanDisk 32GB microSDHC UHS-I card datasheet* [PDF]. SanDisk. Retrieved from https://www.westerndigital.com/content/dam/wdc/website/downloadable_assets/eng/product/memory-cards/sandisk-ultra-microsd/data-sheet-sandisk-ultra-microsd-uhs-i.pdf
- Anker Innovations. (2022). *Anker PowerCore 5000 Portable Charger – Technical specifications* [PDF]. Anker Innovations. Retrieved from https://d2211byn0pk9fi.cloudfront.net/spree/accessories/attachments/8321/PowerCore5000_UserManual.pdf
- Fritzing. (n.d.). *Fritzing – Open-source electronics design tool* [Software]. Fritzing.org. Retrieved from <https://fritzing.org>
- Alphasatronic. (2020). *L293D Motor Driver IC* [Image]. Alphasatronic.lk. Retrieved from <https://alphatronic.lk/wp-content/uploads/2020/09/L293D2520Motor2520Driver2520IC.png>
- Raspberry Pi Foundation. (n.d.). *Raspberry Pi 4 Model B* [Image]. Retrieved from https://tse4.mm.bing.net/th?id/OIP.NdVbjpCW_J0GVTsPt6h7BgHaHa
- IndiaMART. (n.d.). *Small DC Motor (3–9V, 3000 RPM)* [Image]. Retrieved from <https://4.imimg.com/data4/RN/GF/MY-15110710/dc-motor-small-size-3-to-9v-3000rpm-500x500.jpg>
- Amazon. (n.d.). *2WD Smart Car Chassis Kit* [Image]. Retrieved from https://m.media-amazon.com/images/I/71W7+ioVTKL.AC_SL1276.jpg
- Amazon. (n.d.). *Breadboard – 400 Tie Points* [Image]. Retrieved from https://m.media-amazon.com/images/I/61p+QTbMf5L._SL1010_.jpg
- Globalso. (n.d.). *PVC Micro USB Cable* [Image]. Retrieved from <https://cdn.globalso.com/datacablefactory/PVC-Micro-usb-cable.jpg>
- Duracell. (n.d.). *AA Alkaline Battery – Pack of 4* [Image]. Retrieved from http://www.altocraft.com/images/thumbs/0000558_4aa-duracell-alkaline-battery_550.jpeg
- draw.io. (n.d.). *Diagramming software for creating flowcharts and technical drawings*. Retrieved October 12, 2025, from <https://www.draw.io>
- Bilal2866. (n.d.). *ENGG-PROJECT-3 (Week-5-Phase-1)* [Repository]. GitHub. Retrieved October 14, 2025, from <https://github.com/Bilal2866/ENGG-PROJECT-3/tree/Week-5-Phase-1>

14. Appendices

14.1 Full Code Listing

```
#include <wiringPi.h>
#include <softPwm.h>
#include <stdio.h>
#include <string.h>

//
// =====
//      ROBORIDE MOTOR CONTROL PROGRAM
// =====
// This program controls a 2-wheel robot using a Raspberry Pi
// and an L293D motor driver. The robot can move forward,
// backward, and spin left or right using terminal commands.
//
// =====
// Hardware Summary:
//  Controller : Raspberry Pi
//  Driver IC  : L293D
//  Motors    : 2x DC motors (Left & Right)
//  Power     : 5000 mAh power bank (Pi) + 4 AA batteries (motors)
// =====

// ===== Pin Mapping =====
//
// Motor A (Left Side):
//  IN1 → WiringPi 0 → BCM 17 → Physical Pin 11
//  IN2 → WiringPi 1 → BCM 18 → Physical Pin 12
//  ENA → WiringPi 2 → BCM 27 → Physical Pin 13
//
// Motor B (Right Side):
//  IN3 → WiringPi 3 → BCM 22 → Physical Pin 15
//  IN4 → WiringPi 4 → BCM 23 → Physical Pin 16
//  ENB → WiringPi 5 → BCM 24 → Physical Pin 18
//
// Power Connections:
//  Vcc1 (L293D pin 16) → Pi 5V (logic supply)
//  Vcc2 (L293D pin 8) → Motor battery + (~6V)
//  GND (L293D pins 4,5,12,13) → Common ground (Pi GND + Battery -)
//
// =====

// Define pin numbers for Motor A (Left)
#define IN1 0 // GPIO17, Physical Pin 11
#define IN2 1 // GPIO18, Physical Pin 12
#define ENA 2 // GPIO27, Physical Pin 13

// Define pin numbers for Motor B (Right)
#define IN3 3 // GPIO22, Physical Pin 15
#define IN4 4 // GPIO23, Physical Pin 16
#define ENB 5 // GPIO24, Physical Pin 18
```

```
// Speed presets (duty cycles for PWM)
#define SLOW_SPEED 70 // ~70% duty (smooth slow speed)
#define FAST_SPEED 90 // ~90% duty (fast but controllable)

// =====
//           Motor Control Utility Functions
// =====

// Stop both motors completely
void motor_stop() {
    digitalWrite(IN1, LOW);
    digitalWrite(IN2, LOW);
    digitalWrite(IN3, LOW);
    digitalWrite(IN4, LOW);
    softPwmWrite(ENA, 0);
    softPwmWrite(ENB, 0);
    delay(100); // short delay before direction change (safety)
}

// Move the robot forward
void forward(int speed) {
    motor_stop(); // ensure motors are off before changing direction
    digitalWrite(IN1, HIGH);
    digitalWrite(IN2, LOW);
    digitalWrite(IN3, HIGH);
    digitalWrite(IN4, LOW);
    softPwmWrite(ENA, speed);
    softPwmWrite(ENB, speed);
}

// Move the robot backward
void backward(int speed) {
    motor_stop();
    digitalWrite(IN1, LOW);
    digitalWrite(IN2, HIGH);
    digitalWrite(IN3, LOW);
    digitalWrite(IN4, HIGH);
    softPwmWrite(ENA, speed);
    softPwmWrite(ENB, speed);
}

// Spin the robot left in place
void spin_left(int speed) {
    motor_stop();
    // Left motor backward, Right motor forward
    digitalWrite(IN1, LOW);
    digitalWrite(IN2, HIGH);
    digitalWrite(IN3, HIGH);
    digitalWrite(IN4, LOW);
    softPwmWrite(ENA, speed);
    softPwmWrite(ENB, speed);
}
```

```
// Spin the robot right in place
void spin_right(int speed) {
    motor_stop();
    // Left motor forward, Right motor backward
    digitalWrite(IN1, HIGH);
    digitalWrite(IN2, LOW);
    digitalWrite(IN3, LOW);
    digitalWrite(IN4, HIGH);
    softPwmWrite(ENA, speed);
    softPwmWrite(ENB, speed);
}

// =====
//                MAIN PROGRAM
// =====

int main() {
    wiringPiSetup(); // initialize WiringPi library

    // Set all motor control pins as outputs
    pinMode(IN1, OUTPUT);
    pinMode(IN2, OUTPUT);
    pinMode(IN3, OUTPUT);
    pinMode(IN4, OUTPUT);

    // Initialize software PWM on both motor enable pins
    softPwmCreate(ENA, 0, 100); // range: 0–100%
    softPwmCreate(ENB, 0, 100);

    // Variables for user input
    char cmd[10]; // stores user command
    int speed = SLOW_SPEED; // default speed
    int running = 1; // program loop control

    // Display command menu
    printf("=====\n");
    printf("    RoboRide Motor Controller  \n");
    printf("=====\n");
    printf("Commands:\n");
    printf(" FWD = move forward\n");
    printf(" BCK = move backward\n");
    printf(" LFT = spin left\n");
    printf(" RGT = spin right\n");
    printf(" STOP = stop motors\n");
    printf(" 1  = set slow speed (%d%%)\n", SLOW_SPEED);
    printf(" 2  = set fast speed (%d%%)\n", FAST_SPEED);
    printf(" EXIT = quit program\n");
    printf("=====\n");

    // Main control loop
    while (running) {
        printf("\nEnter command: ");
        scanf("%s", cmd);
    }
}
```

```
if (strcmp(cmd, "FWD") == 0) {
    forward(speed);
    printf("Moving forward at %d%% speed.\n", speed);
}
else if (strcmp(cmd, "BCK") == 0) {
    backward(speed);
    printf("Moving backward at %d%% speed.\n", speed);
}
else if (strcmp(cmd, "LFT") == 0) {
    spin_left(speed);
    printf("Spinning left at %d%% speed.\n", speed);
}
else if (strcmp(cmd, "RGT") == 0) {
    spin_right(speed);
    printf("Spinning right at %d%% speed.\n", speed);
}
else if (strcmp(cmd, "STOP") == 0) {
    motor_stop();
    printf("Motors stopped.\n");
}
else if (strcmp(cmd, "1") == 0) {
    speed = SLOW_SPEED;
    printf("Speed set to SLOW (%d%%)\n", speed);
}
else if (strcmp(cmd, "2") == 0) {
    speed = FAST_SPEED;
    printf("Speed set to FAST (%d%%)\n", speed);
}
else if (strcmp(cmd, "EXIT") == 0) {
    motor_stop();
    running = 0;
    printf("Exiting program... Goodbye!\n");
}
else {
    printf("Invalid command! Please try again.\n");
}
}

// Stop motors before closing the program
motor_stop();
return 0;
}
```

14.2 Additional Diagrams or Tables

- L293D H Bridge Driver Schematic: (fig 15)

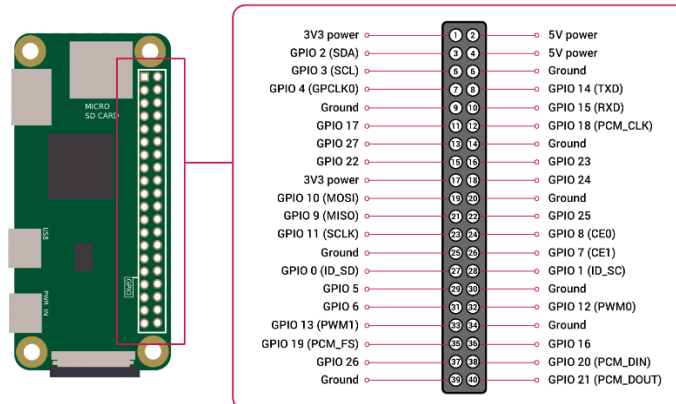


Figure 15: IC Schematic for wiring help

- Footprint for IC: (fig 16)

Footprint

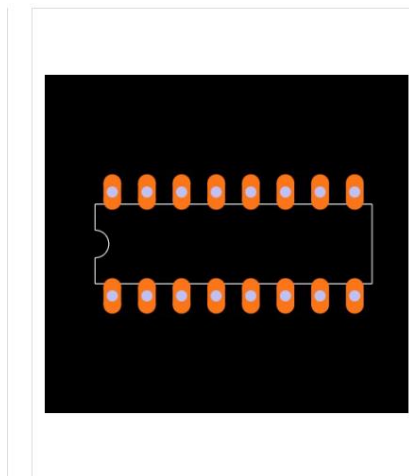


Figure 16: Footprint of L293D IC

- Extra pictures of RoboRide and the group working on it:

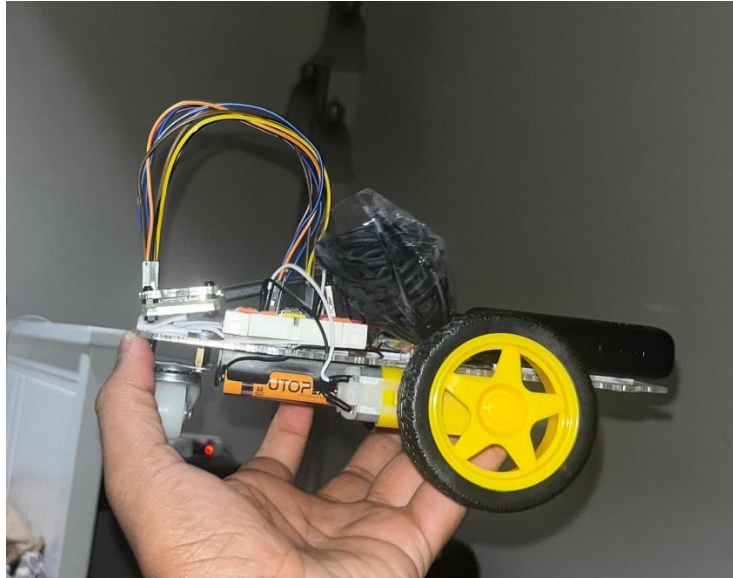
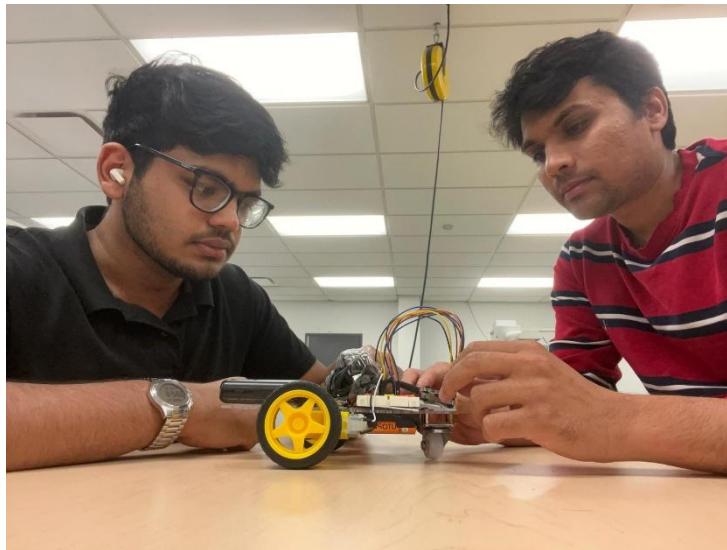


Figure 17: Another angle of RoboRide for better view.



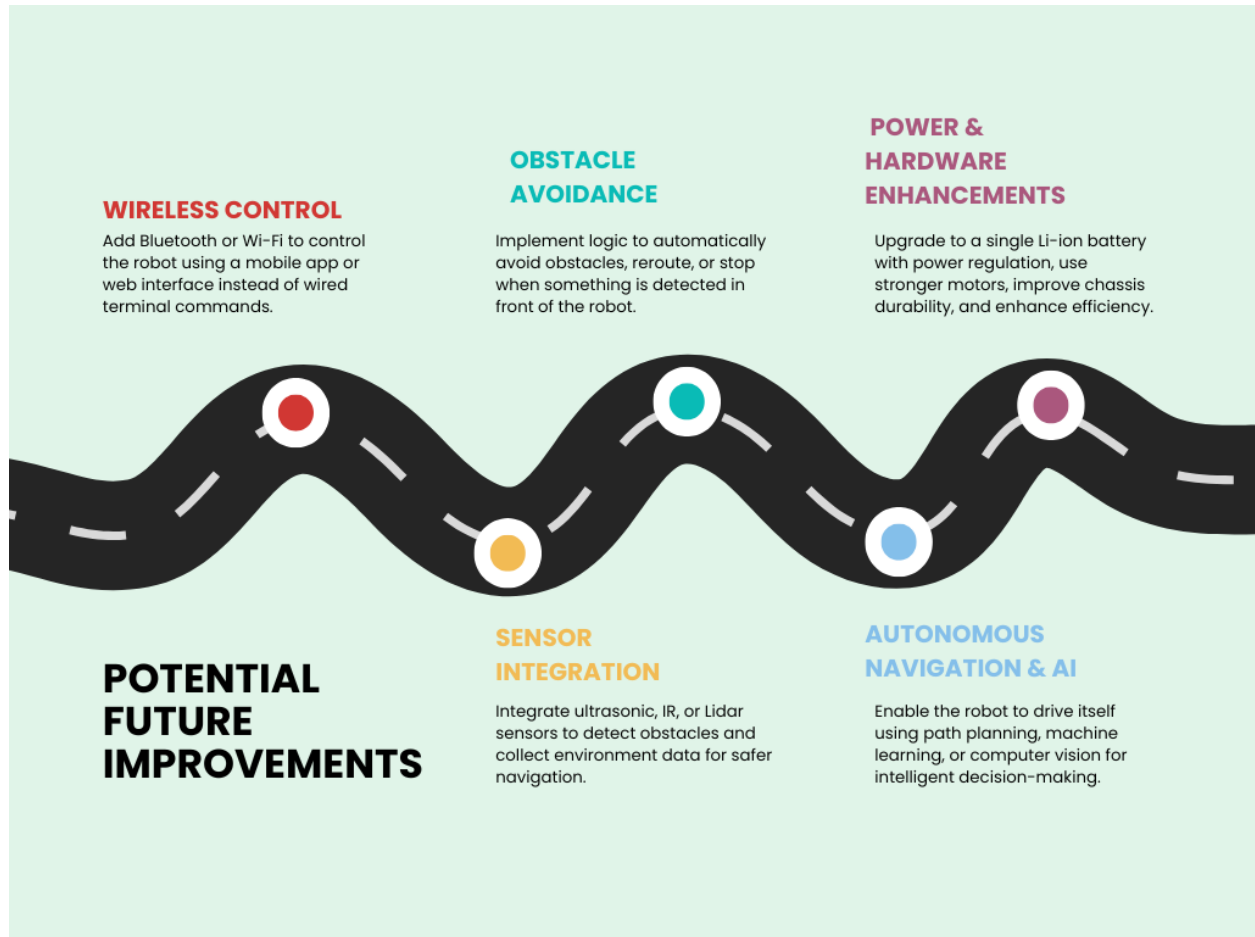


Figure 19: Future Improvements